

L2 - 13.7. Anti-Reverse Engineering

Task 1 - Introduction

Malware authors are constantly looking for ways to evade detection and analysis to maintain the effectiveness of their malware. At the same time, security professionals are working to develop new methods and tools for detecting and mitigating the threat posed by malware.

Reverse engineering is the process of studying a technology product, software, or hardware to learn how it works and extract its functionality or design information. In cybersecurity, reverse engineering is used to understand how malware works, extract indicators of compromise (IOCs), and develop adequate detections, protections, and countermeasures.

As a response, malware authors are motivated to protect their malware from analysts. They use anti-reverse engineering techniques to make malware more challenging to analyze, so it can continue propagating and infecting more systems before security measures are implemented.

This constant back and forth between malware analysts and authors can be aptly described as an "arms race". Malware authors develop new and sophisticated techniques to avoid detection and analysis, and security professionals respond by developing new methods and tools to detect and mitigate the threat posed by malware. As each side grows new techniques, the other responds with even more advanced ones, creating a cycle of escalation.

Task 2 - Anti-Debugging (Overview)

Debugging is the process of examining software to understand its inner workings and identify potential vulnerabilities or issues. Debugging involves software tools called debuggers that allow analysts to step through the code and monitor its execution. Here are the commonly used debuggers for malware analysis nowadays:

- x64dbg

- Ollydbg
- Ida Pro
- Ghidra

Anti-Debugging techniques

Checking for the presence of debuggers	Malware code looks for processes or files associated with debugging tools or hardware-based techniques, such as detecting hardware breakpoints. For example, a common anti-debugging technique uses the Windows API function <code>IsDebuggerPresent</code> to check if a debugger is running.
Tampering with debug registers	Malware may try to modify or corrupt debug logs, which debuggers use to control the execution of code. This prevents the debugger from functioning correctly.
Using self-modifying code	This is a sophisticated technique where malware modifies itself while running, making it difficult for a debugger to follow the code flow.

Answer the questions below

What is the name of the Windows API function used in a common anti-debugging technique that detects if a debugger is running?

IsDebuggerPresent

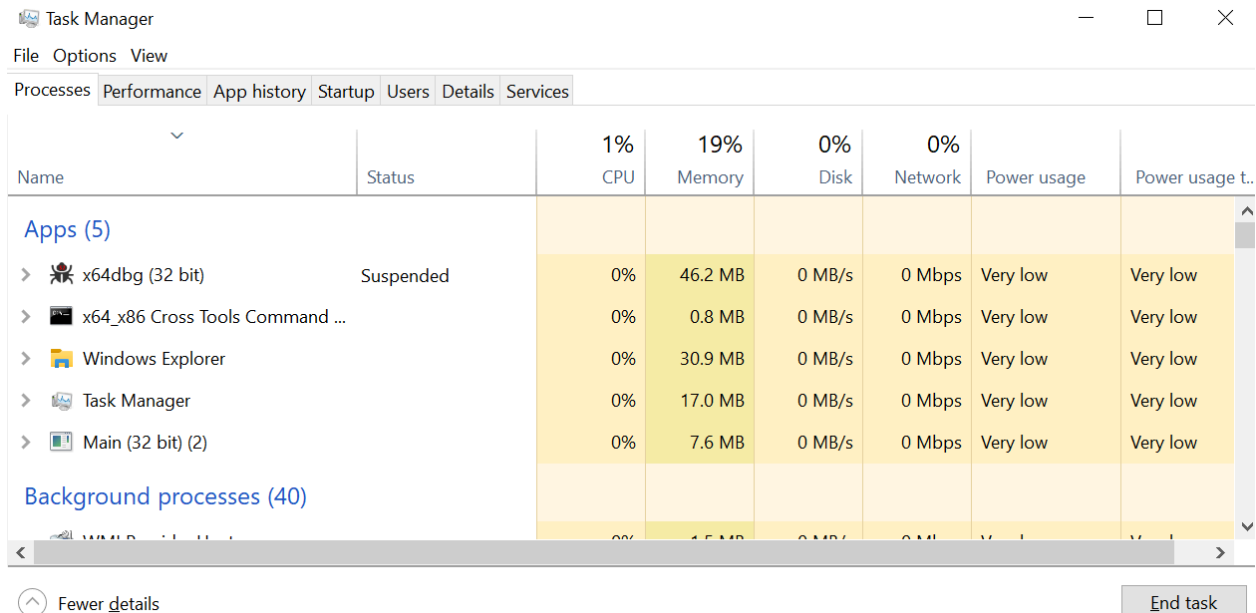
Task 3 - Anti-Debugging using Suspend Thread

Suspend Thread([opens in new tab](#)) is a Windows API function that is used to pause the execution of a thread in a running process. This function has legitimate uses but can also be called from within a malware process to suspend any threads attempting to debug or analyze it. What better way to thwart debugging than not making the debugger work at all?

We have created a simple program that does the above technique.

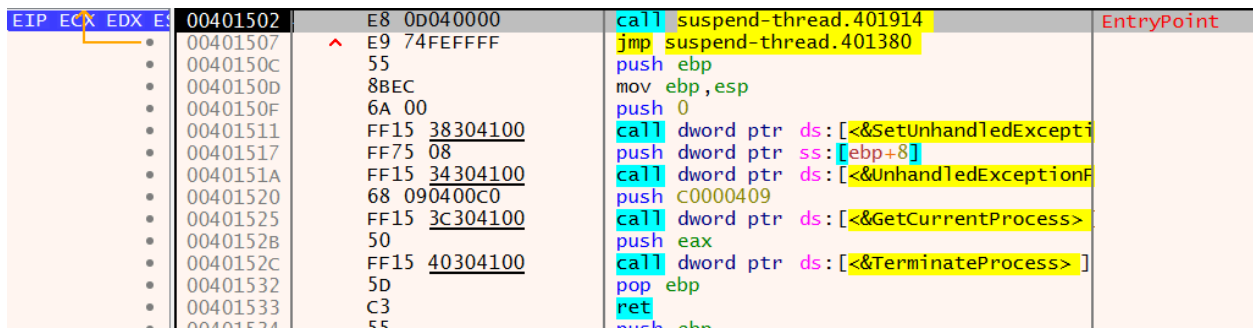
1. Run the x32dbg tool from the Desktop. Open the `suspend-thread.exe` file from the `C:\Malware\` directory.

2. Press **F9** twice to continue the execution of the malware.
3. Notice that x64dbg will now become unresponsive. You can confirm the suspended process by opening Task Manager.



Patching: Patching is one of the most critical skills required of an analyst. In this exercise, we want to patch the function `SuspendThread` so that our debugger won't get suspended.

- Open x32dbg by double-clicking on its icon from the Desktop.
- Press the **F3** key to show the "Open" dialogue window. Look for our sample file called `suspend-thread.exe` in the `C:\Malware\` directory.
- Press the **F9** key once to jump execution to the EntryPoint, as shown below:



- The screenshot shows the 'Calls' window in Immunity Debugger. The window title is 'Calls (suspend-thread.exe)'. It displays a single call instruction at address 004011AB: 'call dword ptr ds:[&SuspendThread]'. The destination is '<kernel32.SuspendThread>'. The search bar at the bottom contains 'SuspendThread'.

Address	Disassembly	Destination
004011AB	call dword ptr ds:[&SuspendThread]	<kernel32.SuspendThread>

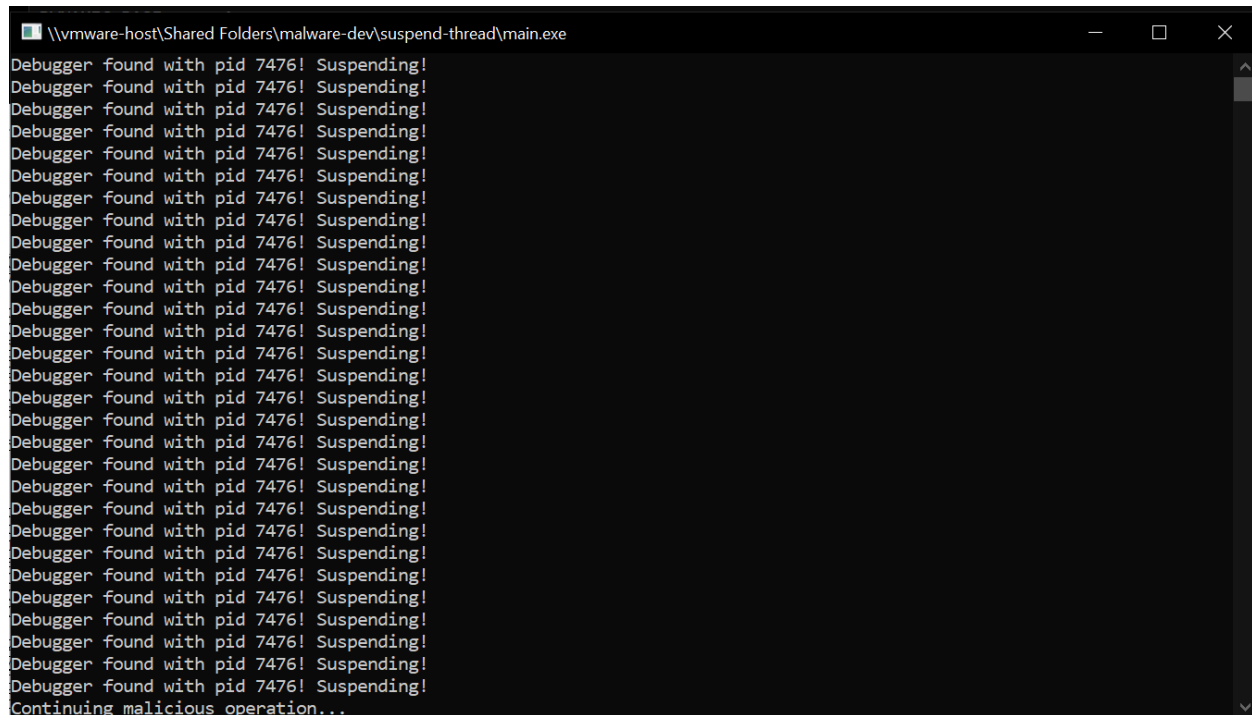
Search: SuspendThread

Module Search 100% Total Progress 100%

- | CPU | Log | Notes | Breakpoints | Memory Map | Call Stack | SEH | Script | Symbols | Source | References | Threads |
|--|-----|-------|-------------|------------|--|-----|--------|---------|--------|------------|---|
| <div> <div>↑</div> <div> <div>004011AB</div> <div>004011AC</div> <div>004011AD</div> <div>004011AE</div> <div>004011AF</div> <div>004011B0</div> <div>004011B1</div> <div>004011B4</div> <div>004011B8</div> <div>004011BA</div> <div>004011C0</div> <div>004011C1</div> <div>004011C6</div> <div>004011CB</div> </div> </div> | | | | | <div> <div>nop</div> <div>nop</div> <div>nop</div> <div>nop</div> <div>nop</div> <div>nop</div> <div>nop</div> <div>mov dword ptr ss:[ebp-30],eax</div> <div>cmp dword ptr ss:[ebp-30],FFFFFFF</div> <div>jne suspend-thread.4011CE</div> <div>call dword ptr ds:[&GetLastError>]</div> <div>push eax</div> <div>push suspend-thread.41A084</div> <div>call suspend-thread.401270</div> <div>add esp,8</div> </div> | | | | | | <div> <div>41A084:"Last error: %i\n"</div> </div> |

- 1

lines that say "Debugger found with PID XXXX! Suspending!", it's not doing anything because we've skipped the function that does the suspending.

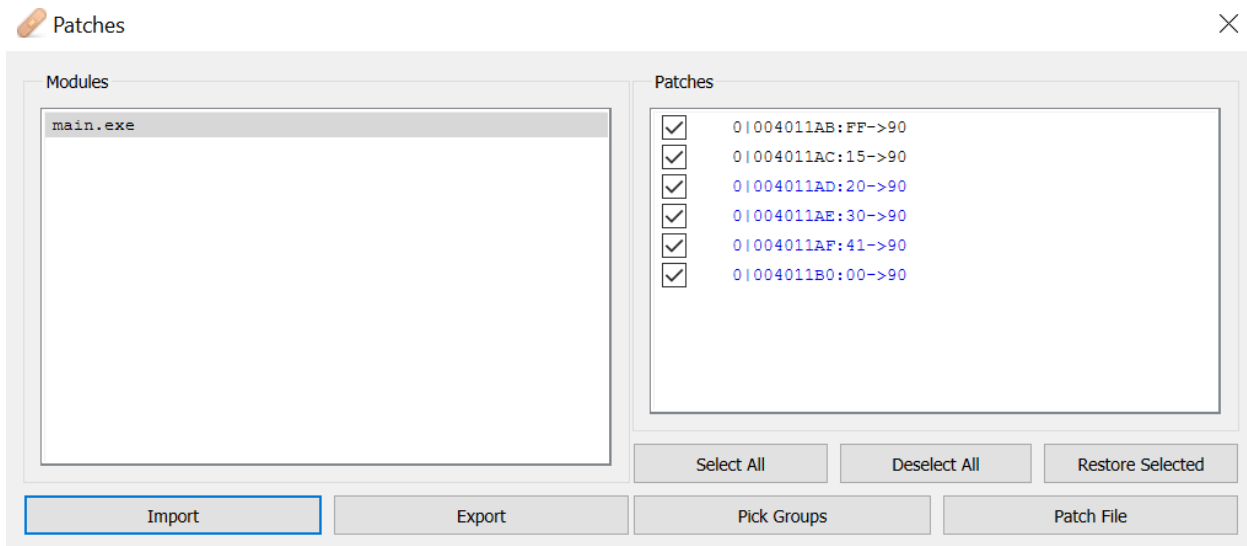


```
\\vmware-host\Shared Folders\malware-dev\suspend-thread\main.exe
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Debugger found with pid 7476! Suspending!
Continuing malicious operation...
```

Exporting and Importing Patches

Stop debugging by pressing **Alt+F2** and go through the patching steps in the previous section again, but do not do the last step.

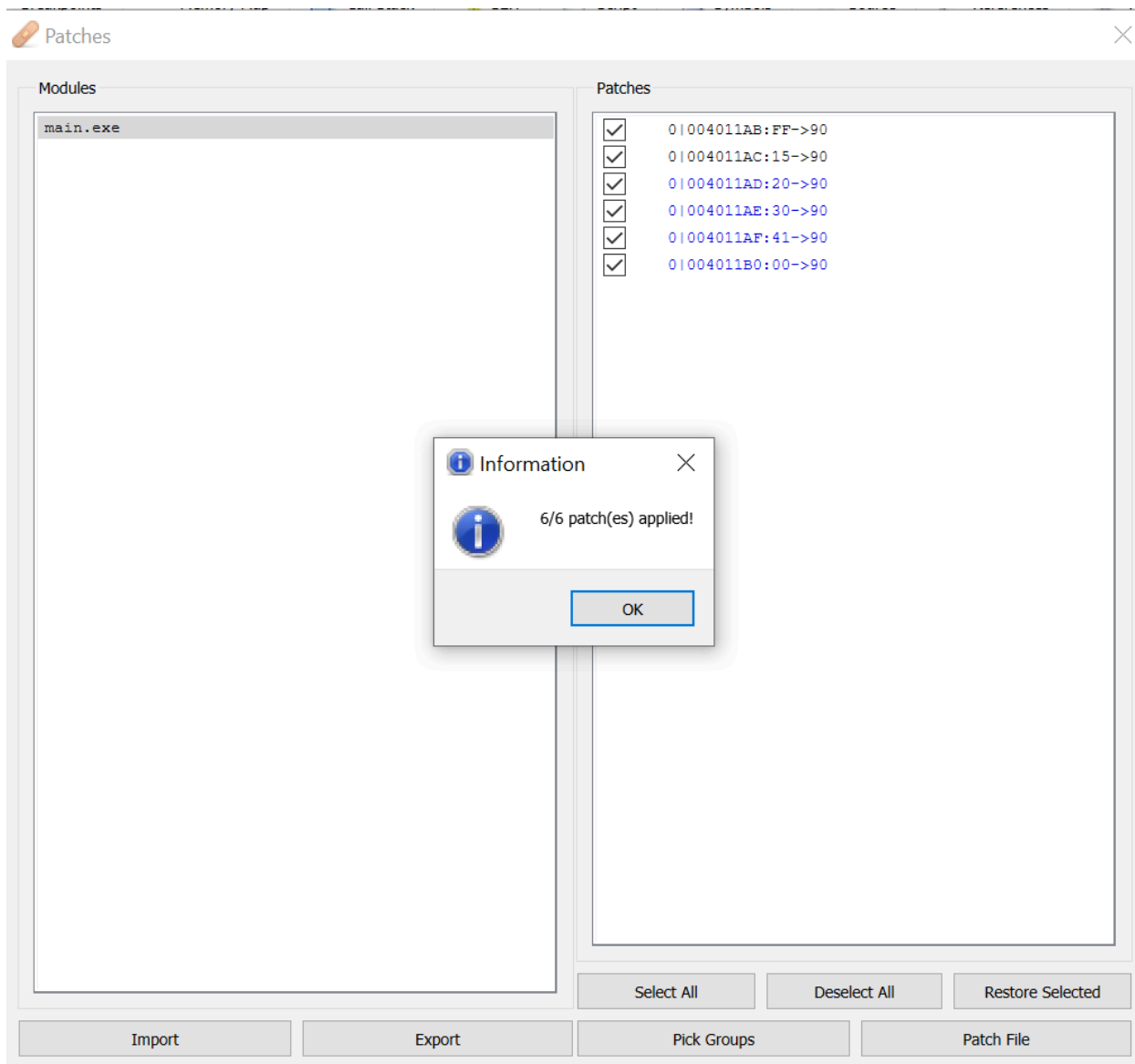
- Once the patches you want are in place, on the top bar, go to "View > Patch file..." to open the "Patch" dialogue window.
- A list of our patches will appear on the right panel. You can see there are six entries. This refers to the six memory locations that we have set to NOPs.



- Click on the "Export" button. On the next dialogue window, click "Yes" to continue. Save this file to a location of your choice.
- Stop the debugger by pressing **Alt+F2**.

Debug the file again. Because debugging has restarted, our previous patches have now been erased. We can restore it by importing the file we made a while ago.

- Go to the patches dialogue window again to "View > Patch file...".
- But this time, press the "Import" button and select the file we saved.



- Once imported, close the window and press **F9** again to continue execution. You'll notice that the previous patches are now re-applied.

Answer the questions below

What is the Windows API function that enumerates windows on the screen so the malware can check the window name?

EnumWindows

What is the hex value of a nop instruction?

What is the instruction found at memory location 004011CB?

add esp,8 (Right-click anywhere on the main screen, select **Go to**, and then click **Expression**)

Task 4 - VM Detection (Overview)

When malware identifies that it is running on a VM, it may decide to respond differently; for example, it may change its behaviour by:

- Executing only a minimal subset of its functionality
- Self-destructing by deleting itself or overwriting parts of its code
- Cause damage to the system by deleting or encrypting files; or
- Not run at all

Detection Techniques

Checking running processes	VMs have easily identifiable processes; for example, VMWare runs a process called <code>vmtoolsd</code> , while VirtualBox has <code>vboxservice</code> . Malware can use the <code>EnumProcess</code> (opens in new tab) Windows API to list all the processes running on the machine and look for the presence of these tools.
Checking installed software	Malware can look in the Windows Registry for a list of installed software under the <code>SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall</code> Registry key. From here, it can check for installed programs like debuggers, decompilers, forensics tools, etc.
Network fingerprinting	Malware can look for specific MAC and network addresses unique to VMs. For example, VMs autogenerate MAC addresses that start with any of the following numbers: <code>00-05-69</code> , <code>00-0c-29</code> , <code>00-1c-14</code> or <code>00-50-56</code> . These numbers are unique and are specifically assigned to a VM vendor called the OUI (Organizationally Unique Identifier).
Checking machine resources	Malware can look at a machine's resources like RAM and CPU Utilization percentages. For example, a machine with RAM amounting to less than 8GB can indicate a lab machine, as they are typically not assigned a significant amount.

Detecting peripherals	Some malware checks for connected printers because this is rarely configured properly on VMs, sometimes not even configured at all.
Checking for domain membership	Corporate networks are a usual target for malware. An easy way to determine this is by checking if the current machine is part of an Active Directory domain. This can quickly be done without the use of API calls by checking the <code>LogonServer</code> and <code>ComputerName</code> environment variables.
Timing-based attacks	Malware can measure the time it takes to execute specific instructions or access particular machine resources. For example, some instructions can be faster on a physical machine compared to a lab machine.

Anti-VM Detection: To prevent malware from using some of the techniques above, we can apply several changes to the system that will remove VM-related artefacts making the VM look less like a VM. For example, we can remove or modify the Registry entries that malware checks for installed programs to hide our debuggers, change the MAC addresses, or configure the VM to appear connected to a printer.

Some researchers have made scripts (See [VMwareCloak\(opens in new tab\)](#) and [VBoxCloak\(opens in new tab\)](#)) to help automate this process. Despite this, even if all the known techniques are addressed, minor architectural checks can still be made by malware, as you will see in the next task.

Answer the questions below

What is the name of the identifiable process used by malware to check if the machine is running inside VirtualBox?

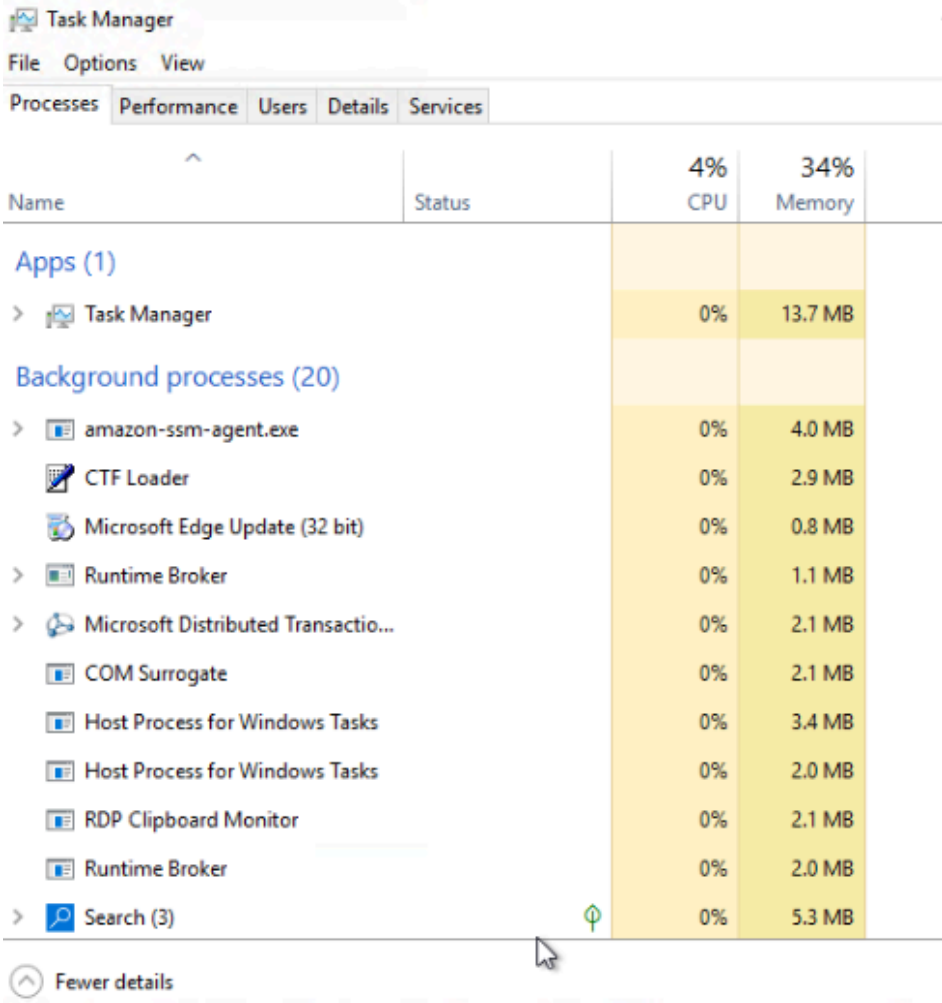
vboxservice

What is the OUI automatically assigned specifically to VMware?

00:50:56 (Check it online)

Using Task Manager, what process indicates that the machine for this room is an Amazon EC2 Lab Machine?

amazon-ssm-agent.exe



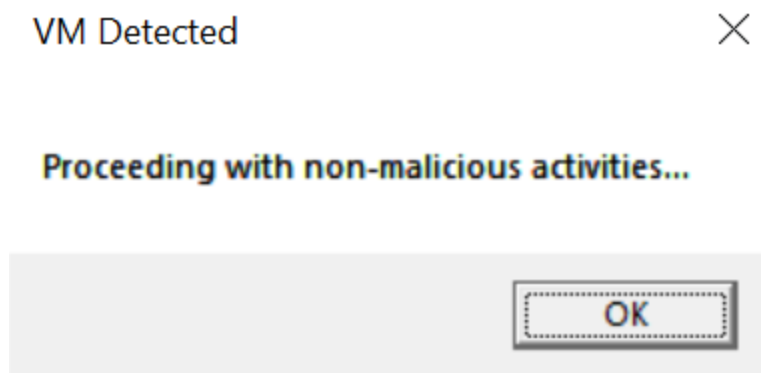
The screenshot shows the Windows Task Manager Performance tab. The CPU usage is 4% and Memory usage is 34%. The list of processes is expanded, showing 'Apps (1)' and 'Background processes (20)'. The 'Task Manager' process is listed under 'Apps (1)' with 0% CPU and 13.7 MB memory. Under 'Background processes (20)', several processes are listed with 0% CPU and varying memory usage: amazon-ssm-agent.exe (4.0 MB), CTF Loader (2.9 MB), Microsoft Edge Update (32 bit) (0.8 MB), Runtime Broker (1.1 MB), Microsoft Distributed Transaction... (2.1 MB), COM Surrogate (2.1 MB), Host Process for Windows Tasks (3.4 MB), Host Process for Windows Tasks (2.0 MB), RDP Clipboard Monitor (2.1 MB), Runtime Broker (2.0 MB), and Search (3) (5.3 MB). A mouse cursor is pointing at the 'Search (3)' process.

Name	Status	4% CPU	34% Memory
Apps (1)			
Task Manager		0%	13.7 MB
Background processes (20)			
amazon-ssm-agent.exe		0%	4.0 MB
CTF Loader		0%	2.9 MB
Microsoft Edge Update (32 bit)		0%	0.8 MB
Runtime Broker		0%	1.1 MB
Microsoft Distributed Transaction...		0%	2.1 MB
COM Surrogate		0%	2.1 MB
Host Process for Windows Tasks		0%	3.4 MB
Host Process for Windows Tasks		0%	2.0 MB
RDP Clipboard Monitor		0%	2.1 MB
Runtime Broker		0%	2.0 MB
Search (3)		0%	5.3 MB

Task 5 - VM Detection by Checking the Temperature

`Win32_TemperatureProbe` is a Windows Management Instrumentation (WMI) class that contains real-time temperature readings from the hardware through the SMBIOS (System Management BIOS) data structure.

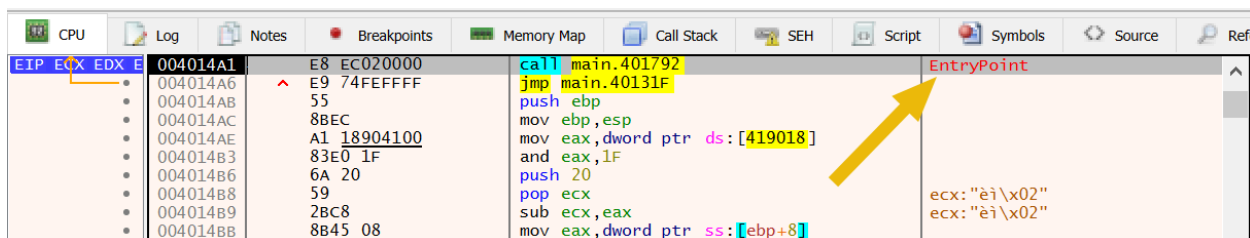
Look for the `vm-detection.exe` file in the `C:\Malware\` directory and execute it. You will see the result immediately.



As you can see, the malware sensed it was running in a VM. It modified its behaviour by continuing a non-malicious activity to throw off analysts snooping around.

Preventing Temperature Checking:

- Open `x32dbg.exe` by double-clicking on the desktop icon
- Go to File > Open, then look for the `vm-detection.exe` file in the `C:\Malware\` directory.
- Press `F9` to start running the program in the debugger. Debugging will halt at the code marked as the "EntryPoint".



- Press `Ctrl+G` on your keyboard to bring up the "Enter expression to follow..." pop-up window. Enter the value `004010E0` and press "OK". This will take you to the memory address. `0x004010E0`.
- Press `F2` to set up a breakpoint at this memory location, then press `F9` once to halt execution here.
- Right now, we are at the memory location `0x004010E0`. The block of instructions from this location up until `0x004010F8` corresponds to this part of our C code snippet: `hr = pEnumerator->Next(WBEM_INFINITE, 1, &pclsObj, &uReturn);`. To help you

visualize this, I've added comments on the right side of the screenshot below to highlight the values passed to the function.

004010E0	8D45 E8	lea eax,dword ptr ss:[ebp-18]	uReturn
004010E3	50	push eax	pcIsObj
004010E4	8D4D F0	lea ecx,dword ptr ss:[ebp-10]	1
004010E7	51	push ecx	WBEM_INFINITE
004010E8	6A 01	push 1	
004010EA	6A FF	push FFFFFFFF	
004010EC	8B55 F8	mov edx,dword ptr ss:[ebp-8]	
004010EF	8B02	mov eax,dword ptr ds:[edx]	
004010F1	8B4D F8	mov ecx,dword ptr ss:[ebp-8]	
004010F4	51	push ecx	
004010F5	8B50 10	mov edx,dword ptr ds:[eax+10]	
004010F8	FFD2	call edx	pEnumerator->Next()

- As previously discussed, this function will fail because we run the program in a VM. After executing, the function will write the value `0` to the memory location `[ebp-18]` (the one marked with `uReturn` in the screenshot above).
- Press `F8` a few times to step through the code until you reach the memory location `004010FD`. Here, we can see that the value of `[ebp-18]` (or `uReturn`) is being compared to zero. This corresponds to this part in the C code: `if (uReturn == 0)`.
- Since we want this "if" condition to evaluate to `False`, what we can do is edit the memory location pointed to by `[ebp-18]` so that it will contain the value `1` instead of `0`.
- To do this, right-click on the line that has `[ebp-18]`, click on "Follow in dump", then select `[ebp-18]`.

004010FD	837D E8 00	cmp dword ptr ss:[ebp-18],0	
00401101	75 07	jne main.40110A	
00401103	33C0	xor eax,eax	
00401105	E9 8D000000	jmp main.401197	
0040110A	6A 00	push 0	
0040110C	6A 00	push 0	
0040110E	8D45 D8	lea eax,dword ptr ss:[ebp-28]	
00401111	50	push eax	
00401112	6A 00	push 0	
00401114	68 68224100	push main.412268	
00401119	8B4D F0	mov ecx,dword ptr ss:[ebp-10]	

- You will see at the bottom of the screen, at the "Dump 1" window tab, that our current memory location is `0019FF08`. This memory location contains the value for `[ebp-18]` or `uReturn`.

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	x= Locals	Struct
Address	Hex				ASCII		
0019FEA8	38 49 7F 00	00 00 00 00	10 FF 19 00	00 00 00 00	8I.....ÿ.....		
0019FEB8	64 1D 7E 00	82 01 BB 6D	C8 EB 7D 44	9B 62 44 30	d.~...»mÊë}D.bd0		
0019FEC8	D0 08 5D E6	78 5B 45 36	60 FF 19 00	90 60 80 70	ð.]æx[E6`ÿ...`p		
0019FED8	FF FF FF FF	20 FF 19 00	FA 00 40 00	C0 49 7F 00	ÿÿÿÿ ÿ...ú.@.ÀI..		
0019FEE8	FF FF FF FF	01 00 00 00	10 FF 19 00	08 FF 19 00	ÿÿÿÿ.....ÿ...ÿ..		
0019FEF8	00 60 23 00	70 00 00 00	09 FC 40 00	E9 01 00 00	.#.p.Éw.ü@.é... ... Å .ÅÅ .0.}		
0019FF08	00 00 00 00	00 00 00 00	C0 C2 7C 00	30 2E 7D 00	ÀI.....(ÿ...@.		
0019FF18	C0 49 7F 00	0C 10 04 80	28 FF 19 00	A8 11 40 00	pÿ....@.....0. .		
0019FF28	70 FF 19 00	19 14 40 00	01 00 00 00	30 05 7C 00	è. ...îÊj.@.j.@.		
0019FF38	E8 95 7C 00	88 00 EC C8	A1 14 40 00	A1 14 40 00	.#.....		
0019FF48	00 60 23 00	00 00 00 00	00 00 00 00	00 00 00 00	<ÿ.....îÿ...ð.@.		
0019FF58	3C FF 19 00	00 00 00 00	CC FF 19 00	F0 1D 40 00	°.Ê.....ÿ...)ú.v		
0019FF68	B0 85 B4 C8	00 00 00 00	80 FF 19 00	29 FA 17 76	.#...ú.vÛÿ...zÇw		
0019FF78	00 60 23 00	10 FA 17 76	DC FF 19 00	9E 7A C7 77			

- Double-click on the rightmost **00** value. This will open up a "Modify Value" window. Enter the value **01** in the expression field and press the "OK" button. The result should look like this:

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	x= Locals	Struct
Address	Hex				ASCII		
0019FEA8	38 49 7F 00	00 00 00 00	10 FF 19 00	00 00 00 00	8I.....ÿ.....		
0019FEB8	64 1D 7E 00	82 01 BB 6D	C8 EB 7D 44	9B 62 44 30	d.~...»mÊë}D.bd0		
0019FEC8	D0 08 5D E6	78 5B 45 36	60 FF 19 00	90 60 80 70	ð.]æx[E6`ÿ...`p		
0019FED8	FF FF FF FF	20 FF 19 00	FA 00 40 00	C0 49 7F 00	ÿÿÿÿ ÿ...ú.@.ÀI..		
0019FEE8	FF FF FF FF	01 00 00 00	10 FF 19 00	08 FF 19 00	ÿÿÿÿ.....ÿ...ÿ..		
0019FEF8	00 60 23 00	70 00 00 00	09 FC 40 00	E9 01 00 00	.#.p.Éw.ü@.é... ... Å .ÅÅ .0.}		
0019FF08	00 00 00 01	00 C2 7C 00	C0 C2 7C 00	30 2E 7D 00	ÀI.....(ÿ...@.		
0019FF18	C0 49 7F 00	0C 10 04 80	28 FF 19 00	A8 11 40 00	pÿ....@.....0. .		
0019FF28	70 FF 19 00	19 14 40 00	01 00 00 00	30 05 7C 00	è. ...îÊj.@.j.@.		
0019FF38	E8 95 7C 00	88 00 EC C8	A1 14 40 00	A1 14 40 00	.#.....		
0019FF48	00 60 23 00	00 00 00 00	00 00 00 00	00 00 00 00	<ÿ.....îÿ...ð.@.		
0019FF58	3C FF 19 00	00 00 00 00	CC FF 19 00	F0 1D 40 00	°.Ê.....ÿ...)ú.v		
0019FF68	B0 85 B4 C8	00 00 00 00	80 FF 19 00	29 FA 17 76	.#...ú.vÛÿ...zÇw		
0019FF78	00 60 23 00	10 FA 17 76	DC FF 19 00	9E 7A C7 77			

- Turn your attention back to the main window (CPU tab) and press **F8** a few times. You'll notice that the code will not exit anymore as it skips the execution from **00401101** to **0040110A**.

What you've done is that you've manipulated the bits of a memory location directly to influence the flow of the program. This method is helpful in many other ways, like for example, you could change the value that will be passed to a function before you execute that function.

However, the program will now crash if you continue running the program. The program was supposed to exit at the previous function, but we've introduced a bug since we intentionally skipped this. Don't worry; this always happens when

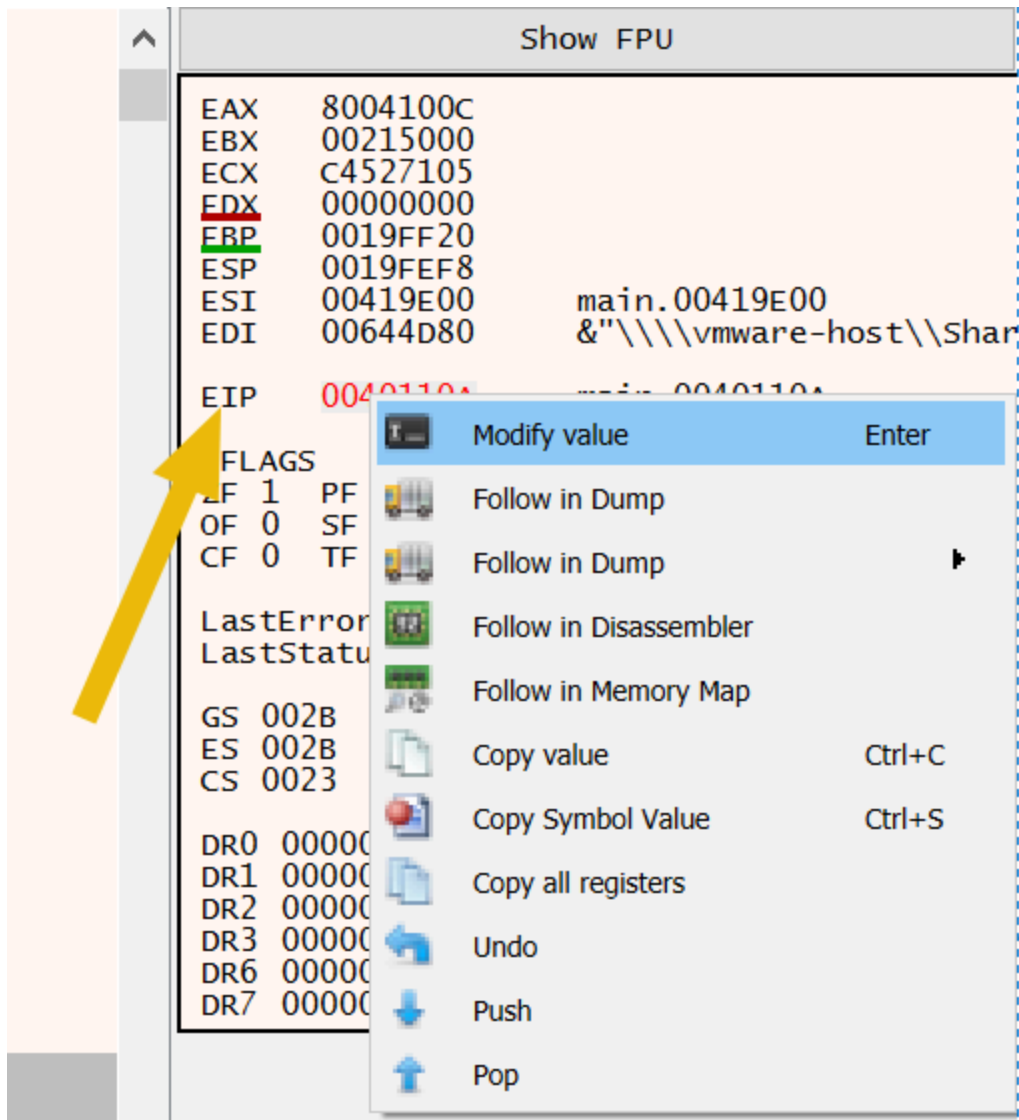
messing around in the debugger. So for us to continue to investigate, there is one other method we can use that would allow us to jump around and continue.

- Let us say we want to skip everything and jump to this part of the code right here: `printf("Thermal Zone Temperature: %d\n", vtProp.intVal);`. In the assembly code, this would be the instructions `00401130` to `00401139`, as shown below:

Address	Disassembly	Comment
00401105	jmp main.401197	Exit the function with 0
0040110A	push 0	
0040110C	push 0	
0040110E	lea eax, dword ptr ss:[ebp-28]	
00401111	push eax	
00401112	push 0	
00401114	push main.412268	412268: "CurrentTemperature"
00401119	mov ecx, dword ptr ss:[ebp-10]	
0040111C	mov edx, dword ptr ds:[ecx]	
0040111E	mov eax, dword ptr ss:[ebp-10]	
00401121	push eax	
00401122	mov ecx, dword ptr ds:[edx+10]	
00401125	call ecx	
00401127	mov dword ptr ss:[ebp-4], eax	
0040112A	cmp dword ptr ss:[ebp-4], 0	
0040112E	j1 main.401148	
00401130	mov edx, dword ptr ss:[ebp-20]	vtProp.intVal
00401133	push edx	
00401134	push main.412290	412290: "Thermal Zone Temperature: %d\n"
00401139	call main.401220	Call to printf()
0040113E	call printf()	
0040113F	add esp, 8	

The great thing about debuggers is that we can change every value related to the program; we can change the values in the memory locations, the values on the stack, and even the values of the registers. The `EIP` register is the register that holds the memory address that tells the debugger what next instruction to execute next. And yes, this can be edited!

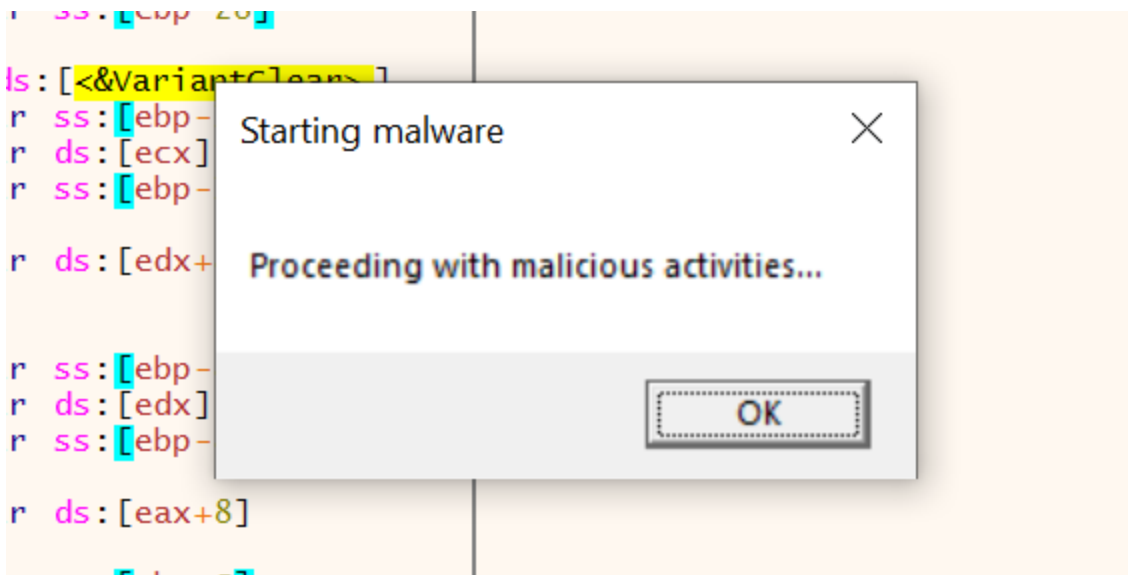
- Continue from the last step. On the "Registers" panel on the right of your screen, look for the one that says `EIP`. Right-click on the value and select "Modify Value".



- Enter `00401134` as the new value and press OK.
- Now when we press `F8` to step through the code, you'll notice that we'll jump directly to that instruction. Notice how the `EIP` marker now points to the location we wanted.

0040110A	6A 00	push 0	
0040110C	6A 00	push 0	
0040110E	8D45 D8	lea eax,dword ptr ss:[ebp-28]	
00401111	50	push eax	
00401112	6A 00	push 0	
00401114	68 68224100	push main.412268	412268:L"CurrentTemperature"
00401119	8B4D F0	mov ecx,dword ptr ss:[ebp-10]	
0040111C	8B11	mov edx,dword ptr ds:[ecx]	
0040111E	8B45 F0	mov eax,dword ptr ss:[ebp-10]	
00401121	50	push eax	
00401122	8B4A 10	mov ecx,dword ptr ds:[edx+10]	
00401125	FFD1	call ecx	
00401127	8945 FC	mov dword ptr ss:[ebp-4],eax	
0040112A	837D FC 00	cmp dword ptr ss:[ebp-4],0	
0040112E	7C 18	j1 main.401148	
00401130	8B55 E0	mov edx,dword ptr ss:[ebp-20]	vtProp.intVal
00401133	52	push edx	
00401134	68 90224100	push main.412290	412290:"Thermal Zone Temperature: %d\n"
00401139	E8 E2000000	call main.401220	Call to printf()
0040113E	837D 08	add esp,8	

- Pressing **F9** again would continue running the program, resulting in the output below:



Answer the questions below

In the C code snippet, what is the full WQL query used to get the temperature from the Win32_TemperatureProbe class?

SELECT * FROM MSAcpi_ThermalZoneTemperature

What register holds the memory address that tells the debugger what instruction to execute next?

EIP

Before uReturn is compared to zero, what is the memory location pointed to by [ebp-4]

0019FF1C (Right-click anywhere on the main screen, select **Go to**, and then click **Expression**)

Task 6 - Packers (Overview)

Obfuscation is a technique that aims to intentionally obscure data and code so that it is harder to understand or analyze. The most common obfuscation techniques used by malware authors include:

- **Using encoding techniques** - This involves encoding data (i.e. command line strings, domain names, etc.) using popular encoding techniques like `XOR` or `Base64`. You may have seen a Base64 encoded strings that look like this `VGhpcyBpcyBhbiBCQVNFNjQgZW5jb2RlZCBzdHJpbmcu==`.
- **Using encryption techniques** - This involves encrypting data such as communications to a command and control server, file formats, and network traffic. The most common types used are symmetric key and public key encryption.
- **Code obfuscation** - This involves various techniques such as manipulating the code to alter its syntax and structure, renaming functions, or splitting code across multiple files or code segments.

Packers is a popular tool for malware authors. There are a lot of Packers available. Each has a unique approach and algorithm for packing. Here is a list of some that were seen used in the wild:

- [Alternate EXE Packer\(opens in new tab\)](#)
- [ASPack\(opens in new tab\)](#)
- [ExeStealth\(opens in new tab\)](#)
- [hXOR-Packer\(opens in new tab\)](#)
- [Milfuscator\(opens in new tab\)](#)
- [MPress\(opens in new tab\)](#)
- [PELock\(opens in new tab\)](#)

- [Themida\(opens in new tab\)](#).
- [UPX: the Ultimate Packer for eXecutables\(opens in new tab\)](#).
- [VMProtect\(opens in new tab\)](#).

It is essential to state that not all packed programs are malicious. Packers are also used by legitimate software to protect their programs, like for protecting their intellectual property from theft. For example, the Packer tool "Themida" is widely used to prevent video game cheating.

Because Packers encrypts and obfuscates a program, it would be impossible to know the malware's capabilities without running it. Because of this, we cannot reliably depend on static analysis and signature-based detection techniques to determine its capabilities. The only information we could glean from a malware sample at this state is the Packer tool used. This can still be a good starting point for an investigation, which we will see in the next task.

Answer the questions below

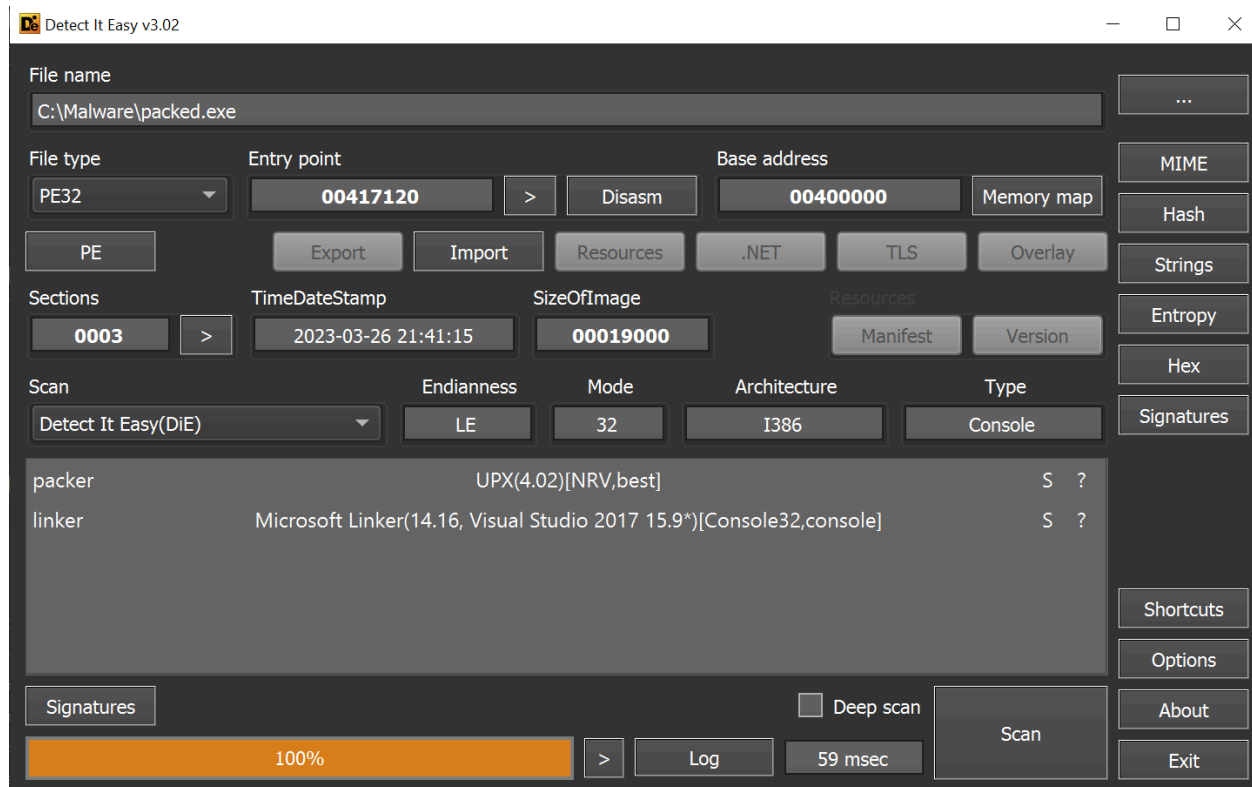
What is the decoded string of the base64 encoded string
"VGhpcyBpcyBhIEJBU0U2NCBibmNvZGVkIHNOcmIuZy4="?

This is a BASE64 encoded string. (use cyber chefr)

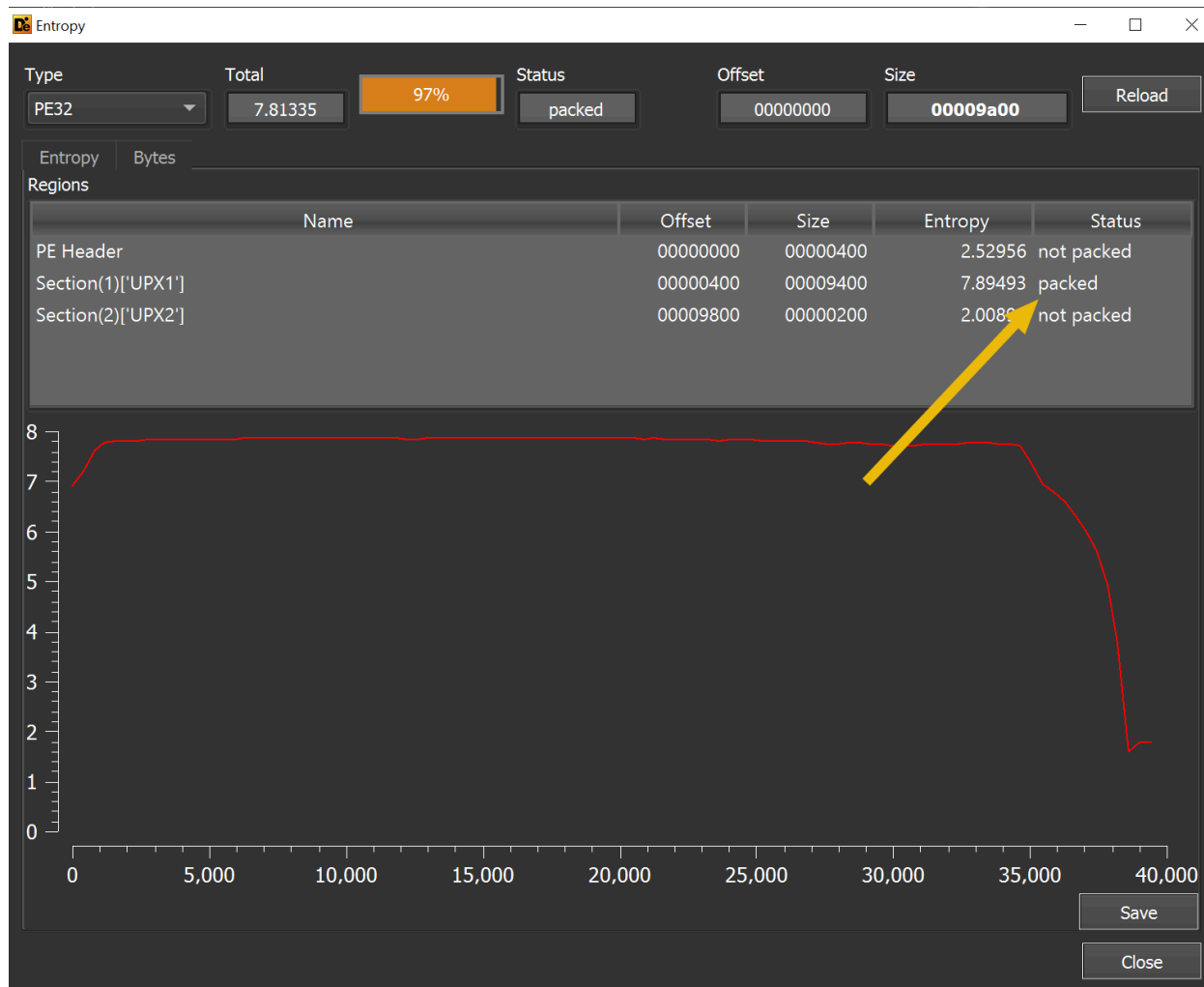
Task 7 - Identifying and Unpacking

The first step in dealing with packed malware is identifying the Packer used. The most accessible tools are DetectItEasy (DIE) and PEStudio, which are already included in the VM attached to this room. You will also find a packed sample program titled `packed.exe` under the `C:\Malware\` directory that we can use to test these tools on. DetectItEasy is a tool that uses a collection of signatures of known packers to identify the one used on the selected file.

1. Right-click on the `packed.exe` executable file and select "Detect it easy (DIE)".
2. If DetectItEasy can identify the used Packer, it will display its "best guess" under this section.



1. You can also click on the "Entropy" button on the right side of the screen to open the "Entropy" window.
2. The Entropy window looks and determines how much entropy each section has. And as you can see, it has been determined that "Section 1" is "packed".



Entropy measures the level of disorder or uncertainty in a system. Packed malware tends to have high entropy due to the randomness of the packing process.

Another tool is PESTudio which lists information on PE files. This is important because there are specific Packers that will still leave clues.

1. From the Desktop, double-click on the "pestudio" icon.
2. Go through the menus, "File > Open", and open the `packed.exe`.
3. In the list of items on the left panel window, click on "sections (self-modifying)".

property	value	value
general		
name	UPX0	UPX1
md5	n/a	2039C9896F5F41EF88FE23E7...
entropy	n/a	7.895
file-ratio (97.40%)	n/a	96.10 %
raw-address	0x00000400	0x00000400
raw-size (38400 bytes)	0x00000000 (0 bytes)	0x00009400 (37888 bytes)
virtual-address	0x00001000	0x0000E000
virtual-size (98304 bytes)	0x0000D000 (53248 bytes)	0x0000A000 (40960 bytes)

Usually, the section names of a standard PE file are `.text`, `.data` and `.rsrc`. But in the output above, we can see that this is the section names have been changed. The strings `UPX0`, `UPX1`, and `UPX2` are one of those identifiable pieces of information that Packers like UPX leave behind. Not all Packers change this value, but usually, this is the most accessible first place to look. Because different Packer tools use various methodologies in packing executables, sometimes tools like DetectItEasy could not identify all types of Packers.

Automated Unpacking: Once the Packer for a packed malware is identified, it is possible to use an unpacker to get back the original file. Some are readily available, like in the case of UPX, where you can use the same packing program for unpacking. For other commercial tools like Themida, you may have to rely on unpacker scripts made by 3rd parties.

Here is a short list of scripts that you could use for specific Unpacker tools:

- [Themida\(opens in new tab\)](#).
- [Enigma Protector\(opens in new tab\)](#).
- [Mpress_unpacker\(opens in new tab\)](#).

Manual Unpacking and Dumping: Ultimately, the best way to unpack malware is to execute it. When a packed malware is executed, the wrapper or container code performs decryption and deobfuscation. Once fully unpacked, the malware can proceed with its true intentions. At this point, the Packer is useless, and we can thoroughly analyze the malware by debugging it while it's in memory using a debugger. We can also dump this unpacked version as a separate executable.

- Open the `packed.exe` using x32dbg.
- Press CTRL+G to open the "Enter expression to follow..." window, input `004172D4` then press "OK".
- You will now be taken to the memory location `004172D4`. This memory location is before the entry point to the legitimate part of the program.
- Press "F2" to set a breakpoint here, then press "F9" twice to move execution to this location.

Address	Disassembly
004172D4	jmp packed.401262
004172D5	add byte ptr ds:[eax],al
004172D6	add byte ptr ds:[eax],ah
004172D7	add byte ptr ds:[eax],al
004172D8	add byte ptr ds:[eax],al
004172D9	add byte ptr ds:[eax],al
004172DA	add byte ptr ds:[eax],al
004172DB	add byte ptr ds:[eax],al
004172DC	add byte ptr ds:[eax],al
004172DD	add byte ptr ds:[eax],al
004172DE	add byte ptr ds:[eax],al
004172DF	add byte ptr ds:[eax],al
004172E0	add byte ptr ds:[eax],al

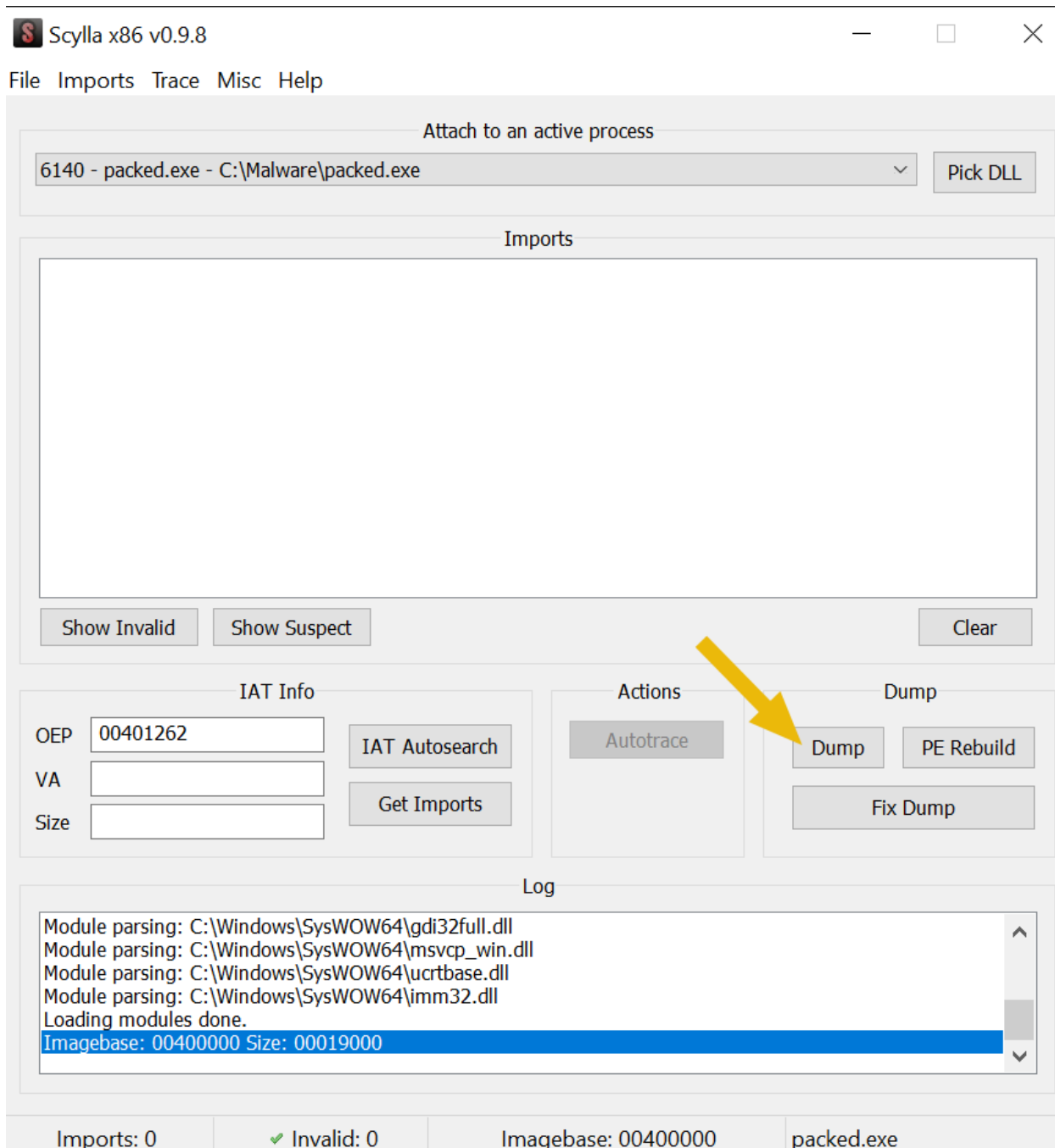
- Finally, press "F7" to jmp to the memory location `00401262`. This location is the starting location of this legitimate program that the Packer goes to once it's fully unpacked.

Address	Disassembly
00401262	call packed.401553
00401263	jmp packed.4010E0
00401264	push ebp
00401265	mov ebp,esp
00401266	mov eax,dword ptr ds:[413028]
00401267	and eax,1F
00401268	push 20
00401269	pop ecx
0040126A	sub ecx,eax
0040126B	mov eax,dword ptr ss:[ebp+8]
0040126C	ror eax,c1
0040126D	xor eax,dword ptr ds:[413028]
0040126E	pop ebp
0040126F	ret
00401270	push ebp

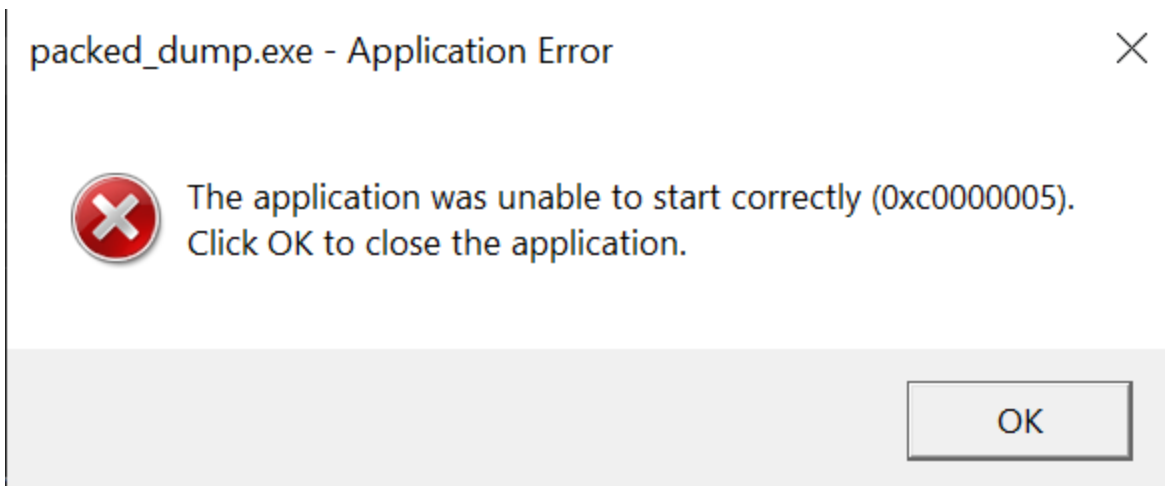
By now, you may be curious about the memory locations `004172D4` and `00401262`, and their relevance to what we are doing. When this UPX-packed malware is executed and the execution location reaches `004172D4`, it indicates that the legitimate part of the program has already been successfully unpacked and is now copied in memory at `00401262`. As outlined in the next steps, we can now begin dumping from this location.

Note: Other packers have different approaches to unpacking the legitimate part of the program, so the steps above would not work on them.

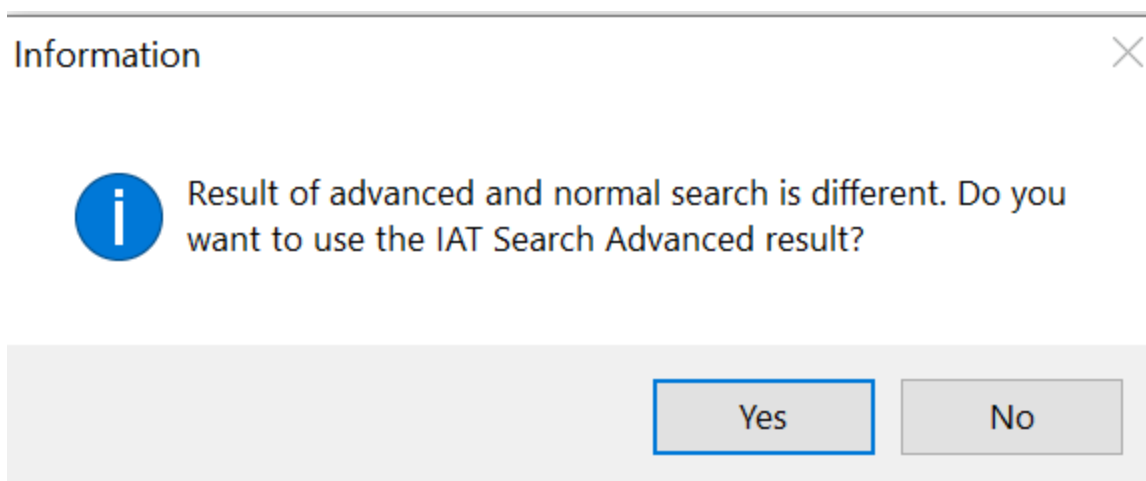
- To start dumping, go to the top bar menu and click "Plugins > Scylla".
- Scylla is a tool that can dump process memory to disk and fix and rebuild the Import Address Table (IAT). We'll use this to dump the unpacked legitimate part of the program to memory and fix it so it will have the updated memory locations from its DLL imports.
- Once the Scylla plugin window is open, click the "Dump" button.



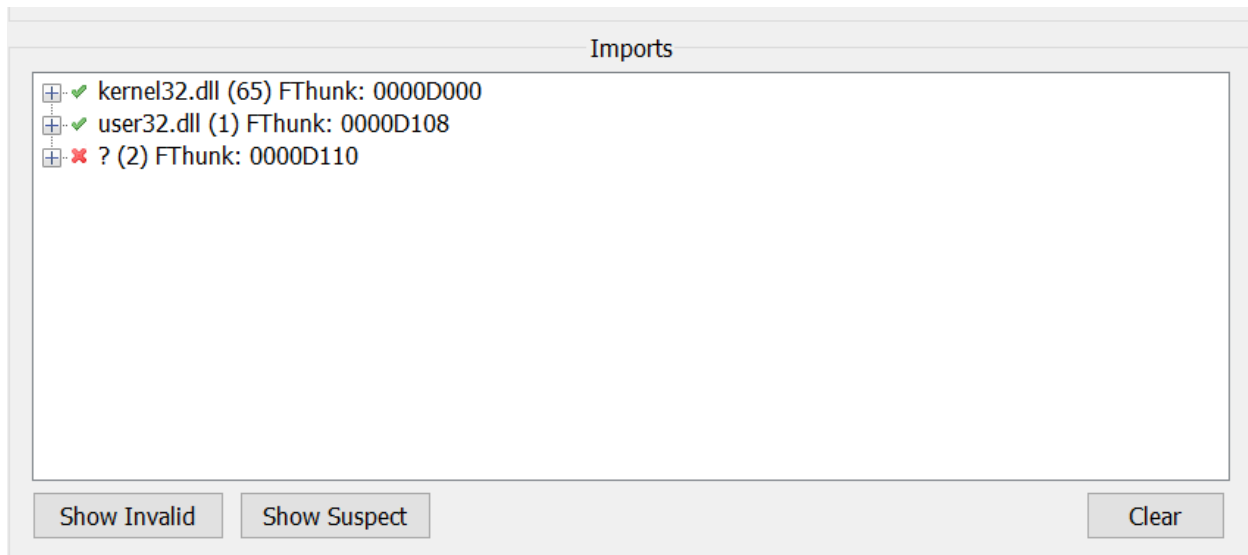
- Save this dumped version to the exact location as the original file.
- If you try to run this dumped version, an error will appear. This is because we have not fixed the import address table yet.



- Back in the Scylla plugin window, and click the "IAT Autosearch" button to start. This scans the memory of the process to locate the import address table.
- If a pop-up window asks if you want to use the IAT Search Advanced result, select "No". Then press "OK".

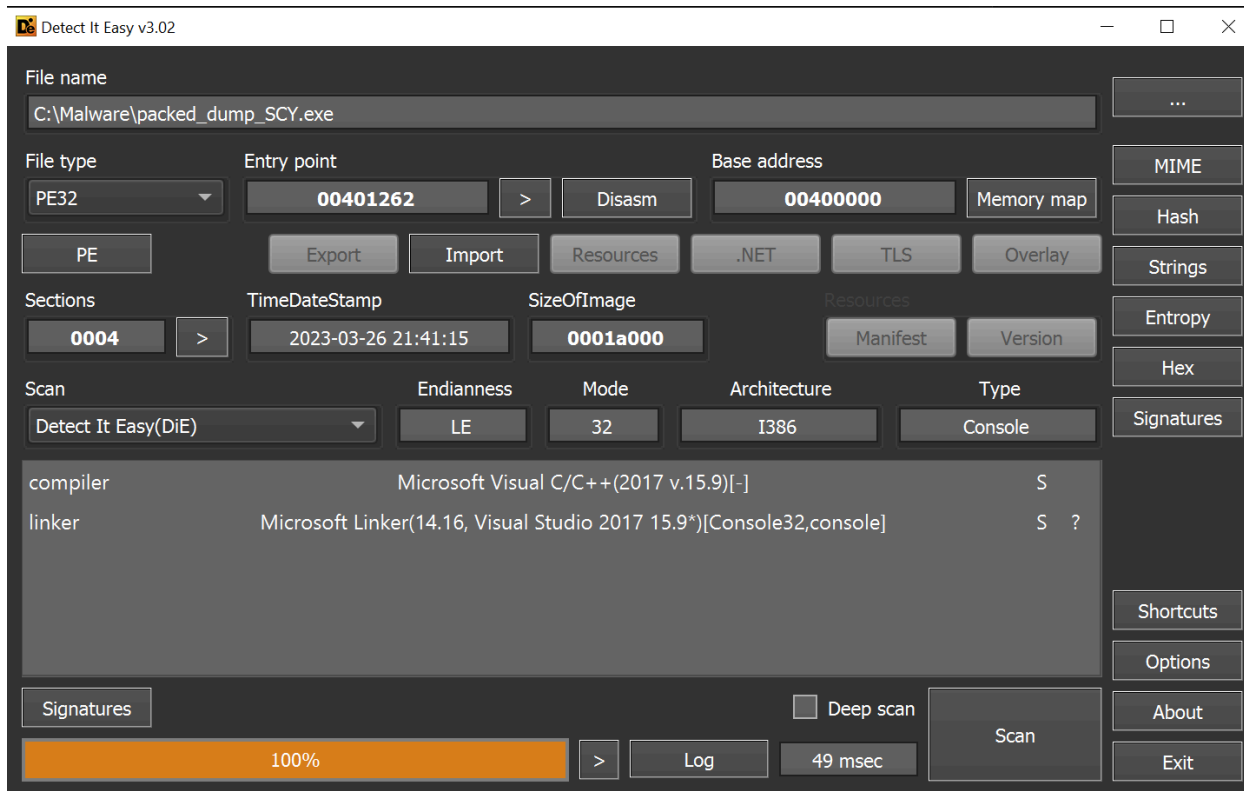


- At this point, the appropriate values related to the IAT are now computed. We can now click on the "Get Imports" button to update the Imports list section.



- This list will show all DLLs that are used by the program. The list is short because the program is simple. An actual malware will have a longer list.
- Sometimes, Scylla will find invalid entries; this is marked with the "X" icon next to it. These are safe to delete because the list already covers every DLL that the program uses. To delete these, go through each invalid entry, right-click, and select "Cut thunk". Once deleted, you should be left with valid entries with check mark icons.
- Click on the "Fix Dump" button, and select the previously generated dump file you created a few steps ago.
- A new file will be created in the same directory as the dumped file with the string "_SCY" appended to its filename. If you execute this, the program will now work correctly.

You can confirm if unpacking was successful by inspecting the generated SCY file using DetectItEasy or PESTudio. Here's the output of DetectItEasy; you'll notice that it does not detect the Packer anymore:



With the malware unpacked, you can now continue investigating the malware. However, don't be fooled into thinking that unpacking packed malware is easy. UPX is the most basic packer tool and the easiest to unpack. Other Packers will be more difficult, primarily if it employs other anti-reverse engineering techniques.

Answer the questions below

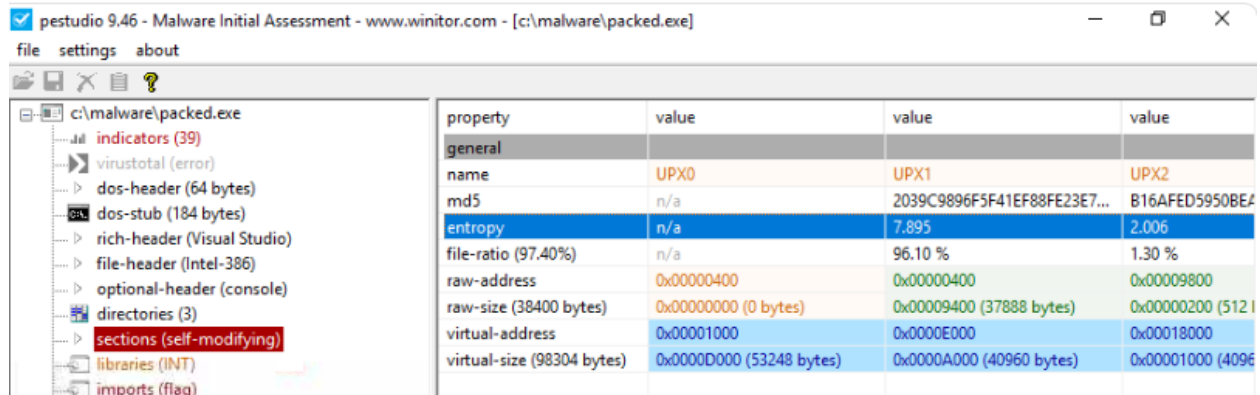
According to DetectItEasy, what is the version of the Microsoft Linker used for linking packed.exe?

14.16

compiler	Microsoft Visual C/C++(2017 v.15.9)[-]	S
linker	Microsoft Linker(14.16, Visual Studio 2017 15.9*)[Console32,console]	S ?

According to pestudio, what is the entropy of the UPX2 section of packed.exe?

2.006



Task 8 - Conclusion

In this room, we learned about the different anti-reverse engineering techniques used by malware today. However, it is crucial to recognize that this topic is expansive, and we've only scratched the surface of what could be explored. Still, we hope this has given you an idea about the techniques and inspired you to explore more.

- Why malware authors use anti-reverse engineering techniques
- About different anti-reverse engineering techniques
- The methods on how to circumvent anti-reverse engineering using various tools
- How anti-reverse engineering techniques are implemented by reading the source code